

**LAPORAN UJIAN AKHIR SEMESTER
PEMROGRAMAN MOBILE LANJUTAN**

**IMPLEMENTASI APLIKASI SMARTKASIR QR
BERBASIS FLUTTER DAN REST API LARAVEL**



**DISUSUN OLEH:
I WAYAN NUGRAHA PURNAMA
NIM: 2323050026**

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS TABANAN
TABANAN
2026**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadapan Tuhan Yang Maha Esa karena laporan Ujian Akhir Semester ini dapat diselesaikan. Laporan ini disusun sebagai dokumentasi implementasi aplikasi SmartKasir QR, yaitu aplikasi kasir berbasis Flutter yang terhubung dengan REST API Laravel, menggunakan autentikasi Bearer Token, database SQLite, serta website pemesanan pelanggan melalui QR meja.

Penyusunan laporan mencakup deskripsi aplikasi, struktur proyek, implementasi fitur, penggunaan endpoint REST API, pengujian, hasil tampilan, serta cara menjalankan sistem. Seluruh tampilan yang disertakan berasal dari aplikasi yang benar-benar dijalankan menggunakan data demo.

Penulis menyadari bahwa sistem dan laporan ini masih dapat dikembangkan. Kritik dan saran yang membangun sangat diharapkan agar aplikasi dapat dikembangkan lebih baik, khususnya pada aspek deployment HTTPS, pembayaran non-tunai, pencetakan struk, serta distribusi Android.

Semoga laporan ini dapat memenuhi tugas Ujian Akhir Semester mata kuliah Pemrograman Mobile Lanjutan dan memberikan gambaran mengenai penerapan Flutter, REST API, autentikasi, serta sinkronisasi data dalam aplikasi kasir.

Tabanan, 17 Juli 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
BAB II DESKRIPSI APLIKASI.....	3
BAB III STRUKTUR PROYEK.....	5
BAB IV IMPLEMENTASI FITUR DAN CODING.....	7
BAB V DAFTAR ENDPOINT REST API.....	18
BAB VI PENGUJIAN.....	19
BAB VII HASIL TAMPILAN.....	21
BAB VIII CARA MENJALANKAN.....	44
BAB IX PENUTUP.....	46

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan produk, pesanan, transaksi, dan stok yang dilakukan secara terpisah dapat menimbulkan keterlambatan pelayanan, kesalahan pencatatan, serta ketidaksesuaian jumlah persediaan. Permasalahan tersebut semakin terasa ketika usaha melayani transaksi kasir sekaligus pemesanan pelanggan dari meja.

SmartKasir QR dikembangkan untuk mengintegrasikan kegiatan kasir dengan pemesanan pelanggan melalui QR Code. Aplikasi Flutter digunakan oleh kasir untuk mengakses dashboard, produk, pesanan, dan transaksi. Laravel menyediakan REST API, autentikasi Bearer Token, website pelanggan, serta logika bisnis yang berkaitan dengan total transaksi dan sinkronisasi stok.

Dengan arsitektur tersebut, data tidak dihitung secara terpisah pada aplikasi pengguna. Harga, total, kembalian, status pesanan, dan perubahan stok tetap divalidasi oleh backend sehingga hasil lebih konsisten dan aman.

1.2 Rumusan Masalah

1. Bagaimana membangun aplikasi kasir Flutter yang terhubung dengan REST API Laravel?
2. Bagaimana menerapkan autentikasi Bearer Token pada aplikasi?
3. Bagaimana mengelola produk, pesanan, transaksi, dan stok dalam satu sistem yang terintegrasi?
4. Bagaimana menerapkan pemesanan pelanggan melalui QR meja dan menghubungkannya dengan aplikasi kasir?

1.3 Tujuan

1. Membangun aplikasi SmartKasir QR berbasis Flutter dan Laravel.
2. Menerapkan login email dan password dengan autentikasi Bearer Token.
3. Menyediakan halaman Home, Bottom Navigation, tiga halaman GET API, dan dua halaman POST API.
4. Mengintegrasikan pemesanan pelanggan melalui QR meja dengan proses kasir dan sinkronisasi stok.
5. Menguji fungsi aplikasi pada sisi Flutter dan Laravel.

1.4 Manfaat

Bagi kasir, aplikasi membantu melihat pesanan, mengelola produk, dan menyelesaikan transaksi dengan alur yang lebih terstruktur. Bagi pelanggan, QR meja mempermudah proses melihat menu dan mengirim pesanan. Bagi pengelola usaha, sistem membantu menjaga konsistensi stok dan riwayat transaksi. Bagi mahasiswa, proyek ini menjadi penerapan nyata materi Flutter, REST API, autentikasi, state management, dan pengujian perangkat lunak.

1.5 Batasan Sistem

- Aplikasi kasir dibuat menggunakan Flutter dan Dart.
- Backend REST API dan website pelanggan dibuat menggunakan Laravel.
- Database menggunakan SQLite.
- Pembayaran yang diterapkan adalah pembayaran tunai.
- Backend dijalankan pada server lokal untuk kebutuhan demonstrasi.
- Sistem belum menggunakan payment gateway, printer struk, push notification, dan deployment publik berbasis HTTPS.

BAB II

DESKRIPSI APLIKASI

2.1 Gambaran Umum SmartKasir QR

SmartKasir QR merupakan sistem kasir terintegrasi yang terdiri atas aplikasi Flutter untuk kasir, REST API Laravel sebagai pusat pengolahan data, website pelanggan yang dibuka melalui QR meja, dan SQLite sebagai database. Aplikasi kasir dan website pelanggan menggunakan data yang sama sehingga pesanan, transaksi, dan stok dapat disinkronkan.

2.2 Pengguna Sistem

Kasir menggunakan aplikasi Flutter untuk login, melihat dashboard, mengelola produk, memproses pesanan, membuat transaksi langsung, menerima pembayaran pesanan QR, dan logout. Pelanggan menggunakan website QR untuk melihat menu, memilih produk, mengirim pesanan, memperoleh nomor pesanan, dan melihat status pesanan.

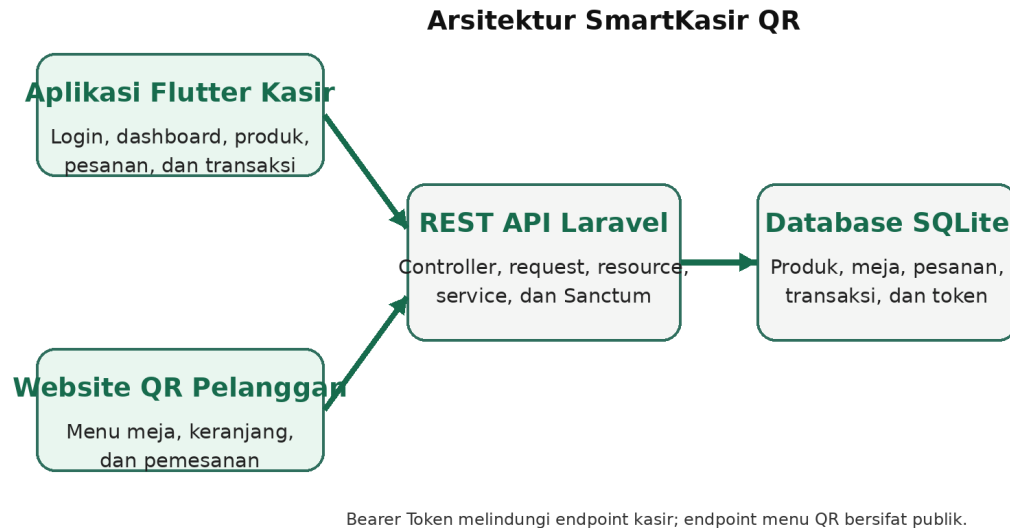
2.3 Teknologi yang Digunakan

Teknologi	Fungsi
Flutter	Membangun aplikasi kasir lintas platform.
Dart	Bahasa pemrograman aplikasi Flutter.
Provider	Mengelola state autentikasi, dashboard, produk, pesanan, dan transaksi.
Dio	Mengirim request HTTP ke REST API Laravel.
flutter_secure_storage	Menyimpan Bearer Token secara aman pada perangkat.
Laravel	Menyediakan REST API, website QR, validasi, dan logika bisnis.
Laravel Sanctum	Menghasilkan dan memvalidasi Bearer Token.
SQLite	Menyimpan data pengguna, produk, meja, pesanan, transaksi, dan token.
Blade/HTML/CSS/JavaScript	Membangun website menu pelanggan melalui QR meja.

2.4 Arsitektur Sistem

Arsitektur sistem memisahkan antarmuka, komunikasi API, logika bisnis, dan penyimpanan data. Flutter berkomunikasi dengan endpoint privat

menggunakan Bearer Token. Website QR pelanggan menggunakan endpoint publik untuk membaca menu dan mengirim pesanan. Laravel memvalidasi request dan menjalankan service sebelum data disimpan ke SQLite.



Gambar 2.1 Arsitektur SmartKasir QR

2.5 Alur Pesanan QR

Pesanan pelanggan dibuat dengan status pending. Kasir kemudian mengonfirmasi pesanan sehingga status berubah menjadi confirmed dan stok dikurangi. Pesanan dapat dilanjutkan menjadi processing, ready, lalu completed setelah pembayaran berhasil. Pesanan berstatus pending, confirmed, processing, atau ready dapat dibatalkan sesuai aturan backend.

pending -> confirmed -> processing -> ready -> completed

pending / confirmed / processing / ready -> cancelled

Stok belum dikurangi ketika pesanan masih pending. Pengurangan stok dilakukan satu kali pada saat confirm. Pembayaran pesanan ready tidak mengurangi stok kembali. Jika pesanan yang stoknya sudah dikurangi kemudian dibatalkan, backend mengembalikan stok satu kali.

BAB III

STRUKTUR PROYEK

3.1 Struktur Folder Flutter

Struktur Utama Flutter

Lokasi file: *smartkasir-qr-mobile/*

```
lib/  
├── config/  
├── core/  
│   ├── errors/  
│   ├── network/  
│   ├── storage/  
│   └── widgets/  
├── features/  
│   ├── auth/  
│   ├── dashboard/  
│   ├── products/  
│   ├── orders/  
│   ├── transactions/  
│   └── profile/  
├── navigation/  
├── app.dart  
└── main.dart  
test/
```

Setiap fitur dipisahkan menjadi model, repository, provider, page, dan widget. Model memetakan data JSON, repository berkomunikasi dengan API, provider mengelola state, sedangkan page dan widget menampilkan antarmuka.

3.2 Struktur Folder Laravel

Struktur Utama Laravel

Lokasi file: *smartkasir-qr-api/*

```
app/  
├── Http/  
│   ├── Controllers/Api/  
│   ├── Requests/  
│   └── Resources/  
├── Models/  
└── Services/  
routes/  
├── api.php  
└── web.php  
database/  
├── migrations/  
└── seeders/  
resources/views/customer/  
tests/
```


Controller menerima request, Form Request menjalankan validasi, Resource membentuk response, Service menangani logika bisnis, Model mengelola data, dan route menentukan endpoint yang dapat diakses.

3.3 Struktur Database

Tabel	Fungsi
users	Data akun kasir dan token autentikasi.
personal_access_tokens	Token Laravel Sanctum.
categories	Kategori produk.
products	Produk, harga, stok, satuan, dan status tersedia.
restaurant_tables	Meja dan token QR.
orders	Nomor pesanan, meja, pelanggan, status, total, dan waktu.
order_items	Item produk dalam pesanan.
transactions	Nomor transaksi, kasir, total, pembayaran, dan kembalian.
transaction_items	Rincian item transaksi.

BAB IV

IMPLEMENTASI FITUR DAN CODING

4.1 Login dan Bearer Token

Kasir memasukkan email dan password pada halaman Login. Data dikirim ke POST /api/login. Setelah berhasil, backend mengembalikan data pengguna dan Bearer Token. Token disimpan menggunakan flutter_secure_storage dan ditambahkan secara otomatis pada header Authorization setiap request privat. Ketika aplikasi dibuka kembali, GET /api/me digunakan untuk memeriksa sesi. Logout memanggil endpoint backend dan selalu menghapus token lokal.

Dio Bearer Token Interceptor

Lokasi file: lib/core/network/api_client.dart

```
_dio.interceptors.add(
  InterceptorsWrapper(
    onRequest: (options, handler) async {
      final token = await tokenStorage.readToken();

      if (token != null && token.isNotEmpty) {
        options.headers['Authorization'] = 'Bearer $token';
      }

      handler.next(options);
    },
  ),
);

final Dio _dio;

/// Penyimpanan token yang dibaca untuk header Authorization.
final TokenStorage tokenStorage;

/// Mengirim request GET dan mengembalikan wrapper response API.
Future<ApiResponseBody> get(
  String path, {
    Map<String, Object?>? queryParameters,
  }) {
  return _request(
    () => _dio.get<Object?>(path, queryParameters: queryParameters),
  );
}
```

AuthRepository Login

Lokasi file: lib/features/auth/repositories/auth_repository.dart

```
/// Melakukan login kasir dan menyimpan token jika berhasil.
///
```

```

/// Parameter [email] dan [password] dikirim ke API login. Mengembalikan
/// UserModel dari response Laravel. Dapat melempar ApiException ketika
/// kredensial salah, validasi gagal, atau server tidak dapat dihubungi.
Future<UserModel> login({
  required String email,
  required String password,
}) async {
  final response = await apiClient.post(
    ApiEndpoints.login,
    data: <String, Object>{'email': email, 'password': password},
  );

  if (!response.success) {
    throw ApiException(message: response.message);
  }

  final data = _asMap(response.data);
  final token = data['token']?.toString();
  final user = _userFromResponse(data['user']);

  if (token == null || token.isEmpty) {
    throw const ApiException(message: 'Token login tidak ditemukan.');
```

TokenStorage

Lokasi file: *lib/core/storage/token_storage.dart*

```

Future<void> delete({required String key}) async {
  _values.remove(key);
}

/// Penyimpanan Bearer Token kasir pada storage aman.
class TokenStorage {
  /// Membuat TokenStorage dengan store yang dapat diganti saat test.
  const TokenStorage(this._store);

  final SecureKeyValueStore _store;

  /// Menyimpan token login kasir secara aman.
  Future<void> saveToken(String token) {
    return _store.write(key: StorageKeys.authToken, value: token);
  }

  /// Membaca token login kasir dan mengembalikan null bila belum login.
  Future<String?> readToken() {
    return _store.read(key: StorageKeys.authToken);
  }

  /// Menghapus token saat logout atau sesi tidak valid.
  Future<void> deleteToken() {
    return _store.delete(key: StorageKeys.authToken);
  }
}
```

```
}
```

```
/// Memeriksa apakah token tersedia di storage aman.
```

AuthProvider

Lokasi file: *lib/features/auth/providers/auth_provider.dart*

```
/// Proses autentikasi gagal dengan pesan error.
failure,
}

/// Provider state autentikasi untuk login, cek sesi, dan logout.
class AuthProvider extends ChangeNotifier {
  /// Membuat provider autentikasi dengan repository yang dapat diganti saat test.
  AuthProvider({required this.authRepository});

  /// Repository yang menangani komunikasi API autentikasi.
  final AuthRepository authRepository;

  AuthStatus _status = AuthStatus.initial;
  UserModel? _currentUser;
  String? _errorMessage;
  bool _hasCheckedAuthentication = false;

  /// Status autentikasi saat ini.
  AuthStatus get status => _status;

  /// User aktif bila sudah authenticated.
  UserModel? get currentUser => _currentUser;

  /// Pesan error terakhir yang aman ditampilkan.
  String? get errorMessage => _errorMessage;

  /// Bernilai true setelah pemeriksaan token awal selesai.
```

4.2 Home Dashboard

Home menggunakan GET `/api/dashboard` untuk menampilkan jumlah pesanan berdasarkan status, jumlah transaksi hari ini, pendapatan hari ini, jumlah produk aktif, stok menipis, dan pesanan terbaru. Dashboard menyediakan refresh dan penanganan error tanpa menampilkan data palsu.

DashboardRepository

Lokasi file: *lib/features/dashboard/repositories/dashboard_repository.dart*

```
///
/// Request menggunakan GET melalui [ApiClient], sehingga header Authorization
/// ditambahkan otomatis bila token tersedia. Mengembalikan [DashboardModel]
/// saat response `success` true dan `data` berupa object. Melempar
/// [ApiException] jika response gagal, data kosong, atau struktur data tidak
/// valid.
```

```

Future<DashboardModel> fetchDashboard() async {
  final response = await apiClient.get(ApiEndpoints.dashboard);

  if (!response.success) {
    throw ApiException(message: response.message);
  }

  final data = response.data;

  if (data is Map<String, dynamic>) {
    return DashboardModel.fromJson(data);
  }

  if (data is Map) {
    return DashboardModel.fromJson(
      data.map((key, value) => MapEntry(key.toString(), value)),
    );
  }

  throw const ApiException(message: 'Data dashboard tidak valid.');
```

4.3 Bottom Navigation

NavigationBar Material 3 menyediakan empat menu utama, yaitu Beranda, Pesanan, Produk, dan Transaksi. IndexedStack mempertahankan state halaman ketika pengguna berpindah tab. Profil dibuka melalui ikon pengguna pada AppBar.

MainNavigationPage

Lokasi file: `lib/navigation/main_navigation_page.dart`

```

import '../features/orders/pages/orders_page.dart';
import '../features/products/pages/products_page.dart';
import '../features/profile/pages/profile_page.dart';
import '../features/transactions/pages/transactions_page.dart';

/// Container navigasi utama dengan destination kasir.
class MainNavigationPage extends StatefulWidget {
  /// Membuat halaman navigasi utama final dengan empat menu UAS.
  const MainNavigationPage({super.key});

  @override
  State<MainNavigationPage> createState() => _MainNavigationPageState();
}

/// State navigasi utama yang menyimpan indeks halaman aktif.
class _MainNavigationPageState extends State<MainNavigationPage> {
  int _selectedIndex = 0;
```

```

static const List<String> _titles = <String>[
  'Beranda',
  'Pesanan',
  'Produk',
  'Transaksi',
];

@override
Widget build(BuildContext context) {
  final user = context.watch<AuthProvider>().currentUser;

```

4.4 GET Produk

Halaman Produk menggunakan GET /api/products dan detail menggunakan GET /api/products/{id}. Fitur pendukung meliputi pencarian, filter kategori, filter ketersediaan, pull-to-refresh, loading, error, dan empty state.

ProductRepository GET

Lokasi file: lib/features/products/repositories/product_repository.dart

```

/// Mengambil daftar produk dari endpoint `/products`.
///
/// Parameter [search], [categoryId], dan [available] dikirim sebagai query
/// hanya bila memiliki nilai. Melempar [ApiException] jika response gagal
/// atau payload data bukan list.
Future<List<ProductModel>> fetchProducts({
  String? search,
  int? categoryId,
  bool? available,
}) async {
  final query = <String, Object?>{};
  final trimmedSearch = search?.trim();

  if (trimmedSearch != null && trimmedSearch.isNotEmpty) {
    query['search'] = trimmedSearch;
  }

  if (categoryId != null) {
    query['category_id'] = categoryId;
  }

  if (available != null) {
    query['available'] = available ? '1' : '0';
  }

  final response = await apiClient.get(
    ApiEndpoints.products,

```

4.5 GET Pesanan

Halaman Pesanan menggunakan GET /api/orders dan GET /api/orders/{id}. Daftar dapat dicari dan difilter berdasarkan status. Detail pesanan

menampilkan meja, pelanggan, item, total, catatan, status stok, serta tombol tindakan yang sesuai dengan status pesanan.

4.6 GET Transaksi

Halaman Transaksi menggunakan GET /api/transactions dan GET /api/transactions/{id}. Data dapat dicari berdasarkan nomor transaksi dan difilter berdasarkan tanggal. Detail menampilkan nomor transaksi, kasir, item, total, pembayaran, kembalian, metode pembayaran, dan waktu.

4.7 POST Tambah Produk

Form Tambah Produk mengirim kategori, nama, SKU, deskripsi, harga, stok, satuan, dan status ketersediaan ke POST /api/products. Validasi lokal mencegah data kosong atau angka negatif, sedangkan Laravel memvalidasi aturan server seperti keunikan SKU. Setelah berhasil, daftar produk dan dashboard diperbarui.

Contoh Request Tambah Produk

Lokasi file: POST /api/products

```
{
  "category_id": 1,
  "name": "Produk Contoh",
  "sku": "PRD-001",
  "description": "Deskripsi produk",
  "price": 5000,
  "stock": 10,
  "unit": "pcs",
  "is_available": true
}
```

ProductRepository POST

Lokasi file: lib/features/products/repositories/product_repository.dart

```
/// Mengirim produk baru ke endpoint `POST /products`.
///
/// [input] berisi data produk yang sudah tervalidasi di UI. Repository hanya
/// mengirim [CreateProductInput.toJson] dan membentuk [ProductModel] dari
/// response. Melempar [ApiException] ketika validasi server gagal atau
/// payload response bukan object.
Future<ProductModel> createProduct(CreateProductInput input) async {
  final response = await apiClient.post(
    ApiEndpoints.products,
    data: input.toJson(),
  );

  if (!response.success) {
```

```

    throw ApiException(message: response.message);
  }

  final data = response.data;
  if (data is! Map) {
    throw const ApiException(message: 'Produk baru tidak valid.');
```

4.8 POST Transaksi Kasir

Transaksi langsung dibuat melalui keranjang pada Flutter. Request hanya mengirim `paid_amount`, `payment_method`, `product_id`, dan `quantity`. Flutter tidak mengirim `harga`, `subtotal`, `total`, `kembalian`, `user_id`, atau nomor transaksi. Backend membaca harga produk, menghitung total dan kembalian, membuat transaksi, lalu mengurangi stok secara atomik.

Contoh Request Transaksi Kasir

Lokasi file: `POST /api/transactions`

```

{
  "paid_amount": 20000,
  "payment_method": "cash",
  "items": [
    {"product_id": 1, "quantity": 2}
  ]
}
```

TransactionRepository POST

Lokasi file: `lib/features/transactions/repositories/transaction_repository.dart`

```

/// Mengirim transaksi langsung kasir ke endpoint `POST /transactions`.
///
/// [input] hanya membawa paid_amount, payment_method, dan product_id beserta
/// quantity. Harga, subtotal, total, kembalian, kasir, dan nomor transaksi
/// dihitung oleh backend. Melempar [ApiException] bila validasi server gagal
/// atau payload response bukan object.
Future<TransactionModel> createTransaction(
  CreateTransactionInput input,
) async {
  final response = await apiClient.post(
    ApiEndpoints.transactions,
    data: input.toJson(),
  );

  if (!response.success) {
    throw ApiException(message: response.message);
  }
}
```



```

final data = response.data;
if (data is! Map) {
  throw const ApiException(message: 'Transaksi baru tidak valid.');
```

```

}

return TransactionModel.fromJson(JsonReaders.asMap(data));
}
```

```

/// Mengirim pembayaran pesanan QR ke endpoint `POST /transactions`.
///
```

4.9 Website Pelanggan Melalui QR

Setiap meja memiliki token QR yang mengarah ke halaman menu pelanggan. Pelanggan dapat mencari produk, memfilter kategori, menambahkan produk ke keranjang, dan mengirim pesanan. Keranjang hanya menyimpan product_id, quantity, dan catatan, sedangkan harga dan total dihitung ulang oleh server.

4.10 Perubahan Status Pesanan

Pesanan pending dapat dikonfirmasi atau dibatalkan. Pesanan confirmed dapat diproses, pesanan processing dapat ditandai siap, dan pesanan ready dapat dibayar. Endpoint PATCH digunakan untuk confirm, process, ready, dan cancel. Flutter tidak mengubah status sebelum backend memberikan response berhasil.

OrderRepository PATCH

Lokasi file: lib/features/orders/repositories/order_repository.dart

```

}
```

```

/// Mengonfirmasi pesanan pending melalui `PATCH /orders/{id}/confirm`.
///
```

```

/// Backend memvalidasi stok dan mengurangnya secara atomik. Mengembalikan
/// [OrderModel] terbaru dengan status `confirmed` bila berhasil.
Future<OrderModel> confirmOrder(int orderId) {
  return _patchOrder(ApiEndpoints.orderConfirm(orderId));
}
```

```

/// Mengubah pesanan confirmed menjadi processing melalui PATCH API.
///
```

```

/// Endpoint tidak mengubah stok. Melempar [ApiException] jika status pesanan
/// sudah berubah atau transisi tidak sah.
Future<OrderModel> processOrder(int orderId) {
  return _patchOrder(ApiEndpoints.orderProcess(orderId));
}
```

```

/// Menandai pesanan processing menjadi ready melalui PATCH API.
///
/// Endpoint tidak mengubah stok dan mengembalikan detail pesanan terbaru.
Future<OrderModel> markOrderReady(int orderId) {
    return _patchOrder(ApiEndpoints.orderReady(orderId));
}

/// Membatalkan pesanan melalui `PATCH /orders/{id}/cancel`.
///
/// [reason] dikirim bila tidak kosong. Backend mengembalikan stok untuk

```

4.11 Pembayaran Pesanan QR

Pembayaran pesanan ready dikirim ke POST /api/transactions menggunakan order_id, paid_amount, dan payment_method. Item tidak dikirim ulang karena backend mengambil item dari pesanan yang tersimpan. Setelah transaksi berhasil, pesanan berubah menjadi completed dan stok tidak dikurangi lagi.

Contoh Request Pembayaran Pesanan

Lokasi file: POST /api/transactions

```

{
  "order_id": 5,
  "paid_amount": 20000,
  "payment_method": "cash"
}

```

4.12 Sinkronisasi Stok

OrderService memastikan stok hanya berkurang sekali ketika pesanan dikonfirmasi. TransactionService membedakan transaksi langsung dengan transaksi dari pesanan. Transaksi langsung mengurangi stok ketika dibuat, sedangkan pembayaran order menggunakan stok yang telah dikurangi pada proses confirm.

OrderService Pengelolaan Stok

Lokasi file: app/Services/OrderService.php

```

*
* @param Order $order Pesanan berstatus pending yang akan dikonfirmasi.
* @return Order Pesanan setelah berhasil dikonfirmasi.
*
* @throws ValidationException Ketika status pesanan atau stok tidak valid.
*/
public function confirm(Order $order): Order
{

```

```

return DB::transaction(function () use ($order): Order {
    $order = Order::query()
        ->with(['restaurantTable', 'items.product'])
        ->whereKey($order->id)
        ->firstOrFail();

    $this->ensureStatus($order, OrderStatus::Pending, 'Pesanan hanya dapat dikonfirmasi saat menunggu.');
```

```

    if ($order->stock_deducted_at === null) {
        $items = $order->items
            ->map(fn ($item) => ['product_id' => $item->product_id, 'quantity' => $item->quantity])
            ->all();

        $this->stockService->deduct($items);
    }

    $order->forceFill([
        'status' => OrderStatus::Confirmed->value,
        'stock_deducted_at' => $order->stock_deducted_at ?? now(),
    ]->save();

```

TransactionService Transaksi

Lokasi file: *app/Services/TransactionService.php*

```

* @param User $cashier Kasir yang berasal dari Bearer Token.
* @param array<string, mixed> $data Data transaksi tervalidasi.
* @return Transaction Transaksi baru dengan relasi lengkap.
*
* @throws ValidationException Ketika stok, status pesanan, atau pembayaran tidak valid.
*/
public function create(User $cashier, array $data): Transaction
{
    if (isset($data['order_id'])) {
        return $this->createFromOrder($cashier, (int) $data['order_id'], (int) $data['paid_amount']);
    }

    return $this->createDirect($cashier, $data);
}

/**
 * Membuat transaksi langsung kasir dan langsung mengurangi stok.
 *
 * @param User $cashier Kasir yang membuat transaksi.
 * @param array<string, mixed> $data Data transaksi langsung tervalidasi.
 * @return Transaction Transaksi langsung yang baru dibuat.
 *
 * @throws ValidationException Ketika stok tidak cukup atau pembayaran kurang.
 */
private function createDirect(User $cashier, array $data): Transaction
{
    return DB::transaction(function () use ($cashier, $data): Transaction {
        $items = $this->stockService->aggregateItems($data['items']);
    });
}

```

4.13 Penanganan State, Error, dan Keamanan

Provider mengatur state initial, loading, loaded, empty, dan failure. Refresh mempertahankan data lama jika request sekunder gagal. ApiException dan ErrorMessageMapper mengubah error teknis menjadi pesan yang aman. Response 401 menghapus token dan mengembalikan aplikasi ke halaman Login. Seluruh provider di-reset ketika logout agar data akun sebelumnya tidak muncul kembali.

Pencegahan submit ganda diterapkan pada login, tambah produk, transaksi, perubahan status pesanan, dan pembayaran. Token tidak disimpan pada source, tidak dicetak pada log, dan tidak ditampilkan pada antarmuka.

Route API Laravel

Lokasi file: routes/api.php

```
use App\Http\Controllers\Api\ProductController;
use App\Http\Controllers\Api\Public\PublicMenuController;
use App\Http\Controllers\Api\Public\PublicOrderController;
use App\Http\Controllers\Api\TransactionController;
use Illuminate\Support\Facades\Route;

Route::post('/login', [AuthController::class, 'login']);

Route::prefix('public')->group(function () { void {
    Route::get('/tables/{qrToken}/menu', [PublicMenuController::class, 'show']);
    Route::post('/tables/{qrToken}/orders', [PublicOrderController::class, 'store']);
    Route::get('/orders/{orderNumber}/status', [PublicOrderController::class, 'status']);
});

Route::middleware('auth:sanctum')->group(function () { void {
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);

    Route::get('/dashboard', DashboardController::class);

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{product}', [ProductController::class, 'show']);
    Route::post('/products', [ProductController::class, 'store']);

    Route::get('/orders', [OrderController::class, 'index']);
    Route::get('/orders/{order}', [OrderController::class, 'show']);
    Route::patch('/orders/{order}/confirm', [OrderController::class, 'confirm']);
```

BAB V

DAFTAR ENDPOINT REST API

Endpoint berikut diaudit dari routes/api.php. Endpoint privat memakai middleware auth:sanctum dan membutuhkan Bearer Token, sedangkan endpoint menu dan pemesanan pelanggan bersifat publik.

Method	Endpoint	Auth	Fungsi	Halaman
POST	/api/login	Tidak	Login dan menghasilkan Bearer Token.	Login
GET	/api/me	Bearer	Memeriksa sesi pengguna.	Pemulihan sesi
POST	/api/logout	Bearer	Logout dan menghapus token server.	Profil
GET	/api/dashboard	Bearer	Mengambil ringkasan dashboard.	Beranda
GET	/api/categories	Bearer	Mengambil kategori produk.	Produk
GET	/api/products	Bearer	Mengambil daftar produk dan filter.	Produk
GET	/api/products/{id}	Bearer	Mengambil detail produk.	Detail Produk
POST	/api/products	Bearer	Menyimpan produk baru.	Tambah Produk
GET	/api/orders	Bearer	Mengambil daftar pesanan.	Pesanan
GET	/api/orders/{id}	Bearer	Mengambil detail pesanan.	Detail Pesanan
PATCH	/api/orders/{id}/confirm	Bearer	Konfirmasi pesanan dan kurangi stok.	Detail Pesanan
PATCH	/api/orders/{id}/process	Bearer	Mengubah pesanan menjadi diproses.	Detail Pesanan
PATCH	/api/orders/{id}/ready	Bearer	Menandai pesanan siap.	Detail Pesanan
PATCH	/api/orders/{id}/cancel	Bearer	Membatalkan dan mengembalikan stok bila perlu.	Detail Pesanan
GET	/api/transactions	Bearer	Mengambil daftar transaksi.	Transaksi
GET	/api/transactions/{id}	Bearer	Mengambil detail transaksi.	Detail Transaksi
POST	/api/transactions	Bearer	Membuat transaksi langsung atau pembayaran order.	Transaksi/Pembayaran
GET	/api/public/tables/{qrToken}/menu	Tidak	Mengambil menu publik berdasarkan QR meja.	Website Pelanggan
POST	/api/public/tables/{qrToken}/orders	Tidak	Membuat pesanan pelanggan.	Website Pelanggan
GET	/api/public/orders/{orderNumber}/status	Tidak	Melihat status pesanan publik.	Website Pelanggan

BAB VI

PENGUJIAN

6.1 Pengujian Flutter

- dart format: lulus, 0 perubahan.
- flutter analyze: No issues found.
- flutter test: 89 tests passed.

6.2 Pengujian Laravel

- php artisan test: 95 tests passed.
- Assertions: 267.
- Laravel Pint: passed.
- npm run build: passed.

6.3 Tabel Hasil Pengujian

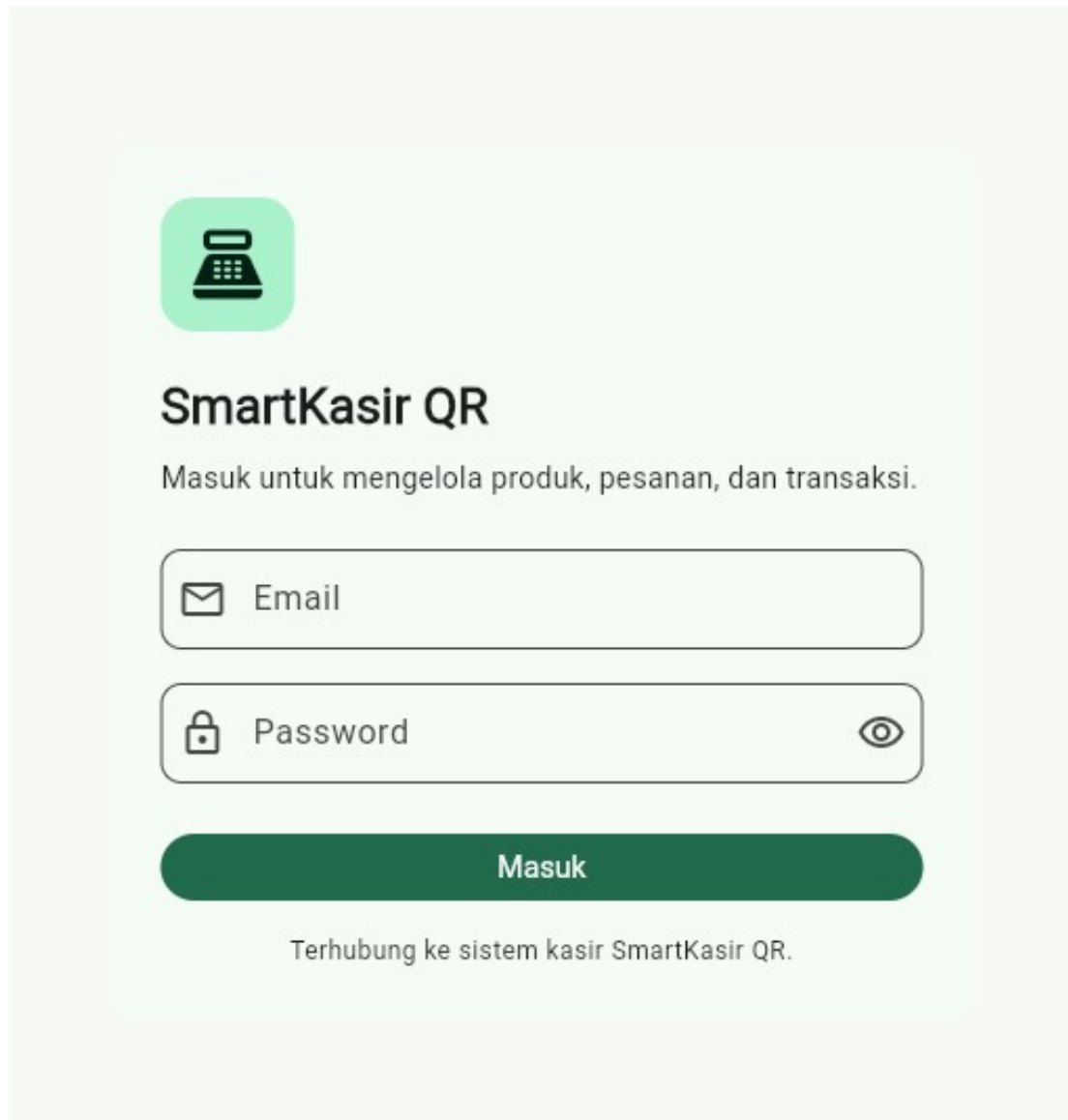
Skenario	Hasil yang Diharapkan	Hasil Aktual	Status
Login benar	Pengguna dapat masuk dengan akun demo.	Login Flutter ke API berhasil.	Lulus
Login salah	Aplikasi menampilkan pesan kesalahan.	Pesan email atau password salah tampil.	Lulus
Pemulihan sesi	Token diverifikasi melalui GET /api/me.	Sesi pulih setelah aplikasi dimuat ulang.	Lulus
Logout	Token dihapus dan kembali ke Login.	Logout API dan lokal berhasil.	Lulus
GET Dashboard	Ringkasan dashboard tampil.	Data nyata dari API tampil.	Lulus
GET Produk	Daftar dan detail produk tampil.	Search, filter, dan detail bekerja.	Lulus
GET Pesanan	Daftar dan detail pesanan tampil.	Filter status dan item bekerja.	Lulus
GET Transaksi	Daftar dan detail transaksi tampil.	Search, filter tanggal, dan detail bekerja.	Lulus
POST Produk	Produk baru tersimpan.	Produk berhasil dibuat dan daftar diperbarui.	Lulus
SKU duplikat	Server menolak SKU yang sama.	HTTP 422 dan pesan SKU sudah digunakan.	Lulus
POST Transaksi	Transaksi tersimpan dan stok berkurang.	Nomor transaksi, total, bayar, dan kembalian benar.	Lulus
Pembayaran kurang	Server menolak pembayaran kurang.	Validasi lokal dan server bekerja.	Lulus
Pesanan QR pending	Pesanan dibuat tanpa mengurangi stok.	Status pending dan stok tetap.	Lulus
Konfirmasi pesanan	Status confirmed dan stok berkurang.	Stok berkurang satu kali.	Lulus
Proses pesanan	Status menjadi processing.	Perubahan status berhasil.	Lulus
Pesanan siap	Status menjadi ready.	Perubahan status berhasil.	Lulus

Skenario	Hasil yang Diharapkan	Hasil Aktual	Status
Pembayaran order	Order completed dan transaksi dibuat.	Pembayaran berhasil dan stok tidak berkurang ulang.	Lulus
Pembayaran ganda	Pembayaran kedua ditolak.	Server menolak dengan HTTP 422.	Lulus
Cancel pending	Pesanan dibatalkan tanpa perubahan stok.	Stok tetap.	Lulus
Cancel confirmed	Pesanan dibatalkan dan stok kembali.	Stok dikembalikan satu kali.	Lulus
401 Unauthorized	Sesi dihapus dan kembali ke Login.	Auth flow menangani token invalid.	Lulus
Server tidak tersedia	UI menampilkan pesan koneksi.	Lulus melalui automated test menggunakan fake repository/provider.	Lulus

BAB VII

HASIL TAMPILAN

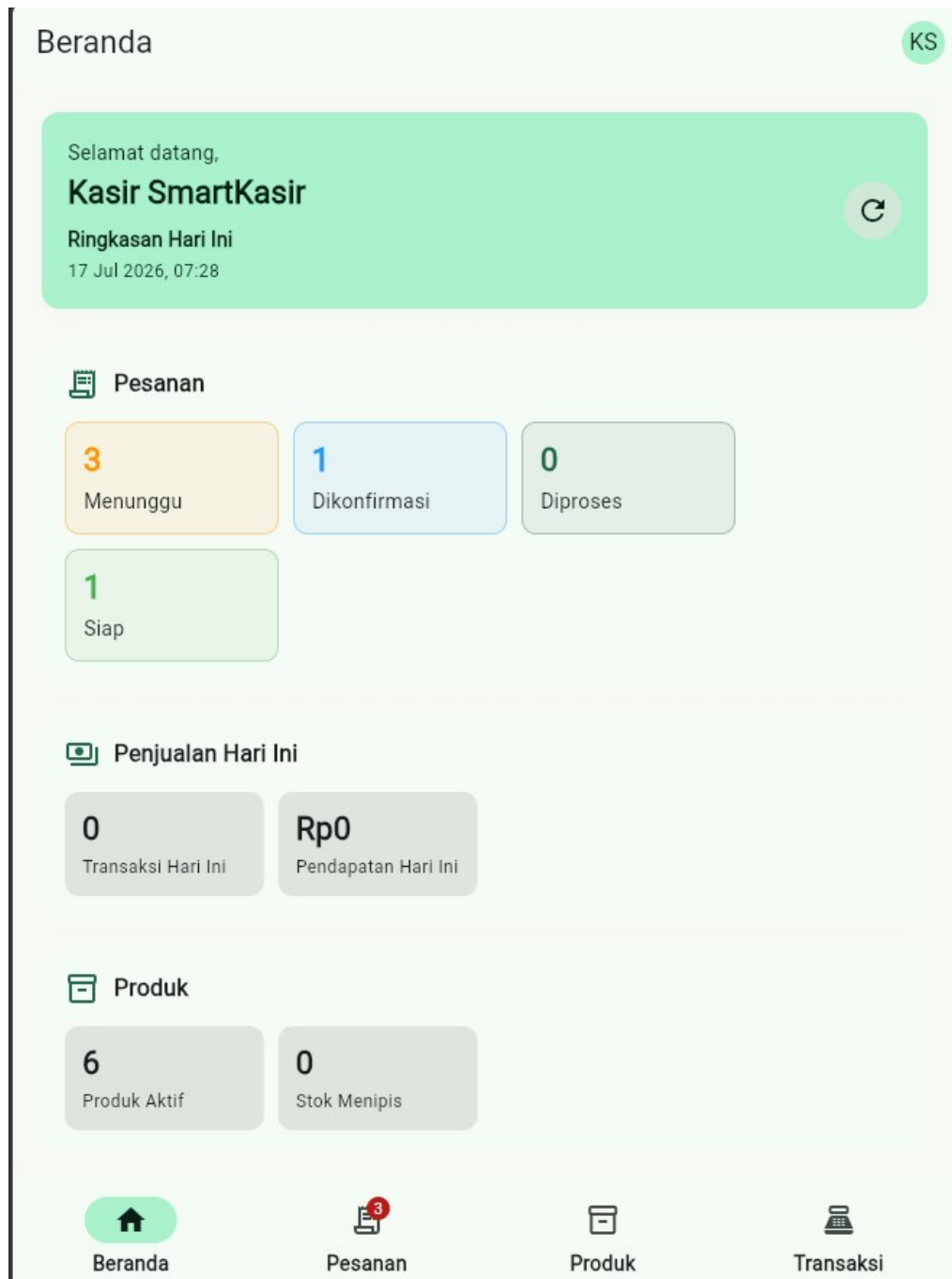
Bagian ini menampilkan hasil antarmuka yang diperoleh dari aplikasi SmartKasir QR dan website pelanggan. Screenshot menggunakan data demo dan tidak menampilkan token, password, atau informasi rahasia.



The image shows a login interface for 'SmartKasir QR'. At the top, there is a green square icon containing a black cash register. Below the icon, the text 'SmartKasir QR' is displayed in a bold, black font. Underneath the title, a subtitle reads 'Masuk untuk mengelola produk, pesanan, dan transaksi.' in a smaller, regular black font. The login form consists of two input fields: the first is labeled 'Email' with an envelope icon on the left, and the second is labeled 'Password' with a lock icon on the left and an eye icon on the right to toggle visibility. Below these fields is a large, dark green button with the white text 'Masuk'. At the bottom of the form, a message states 'Terhubung ke sistem kasir SmartKasir QR.'

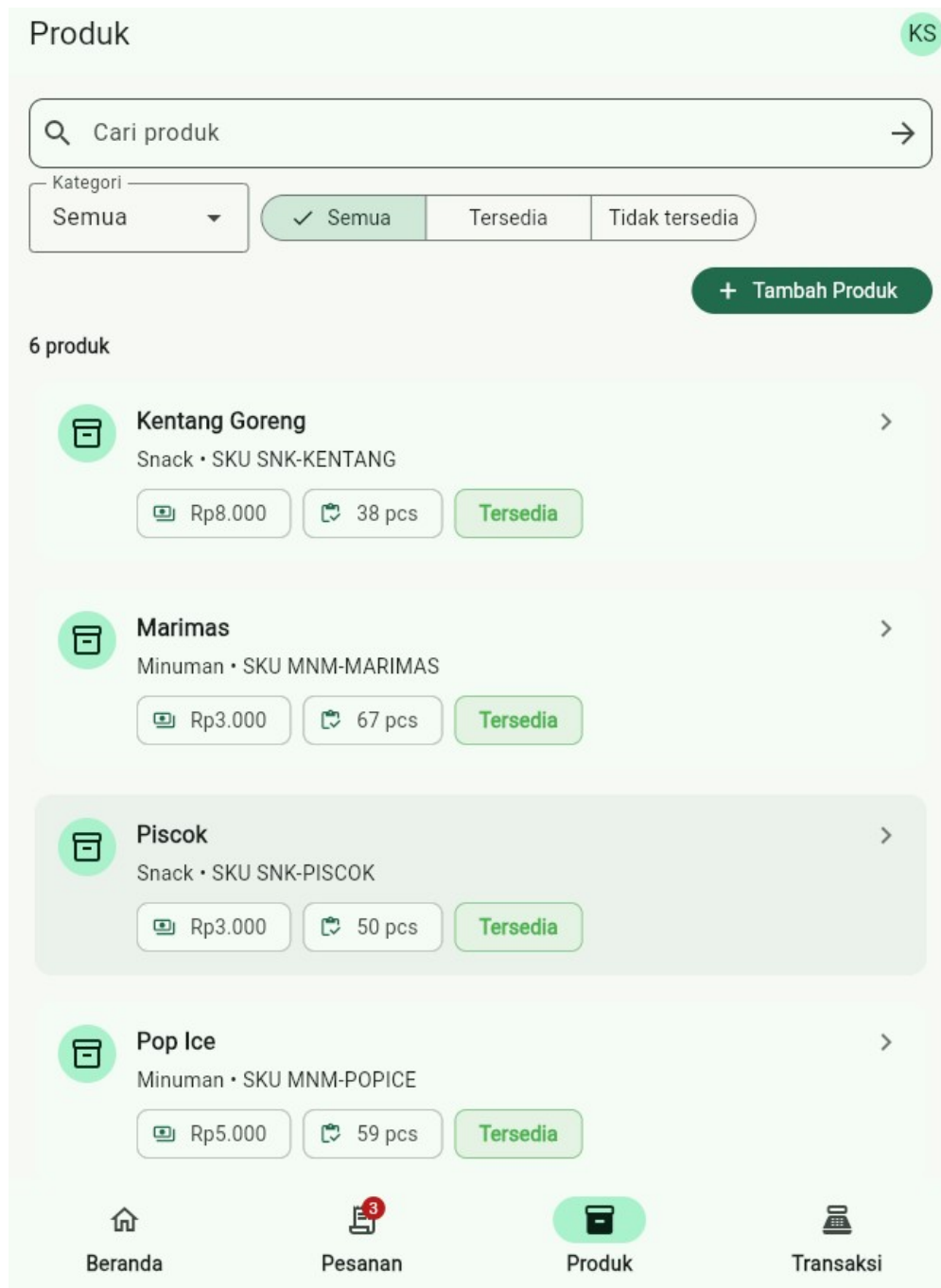
Gambar 7.1 Halaman Login

Halaman Login digunakan kasir untuk memasukkan email dan password. Tombol Masuk dilindungi dari submit ganda dan pesan kesalahan ditampilkan dengan bahasa yang mudah dipahami.



Gambar 7.2 Home Dashboard

Dashboard menampilkan ringkasan pesanan, penjualan hari ini, produk aktif, stok menipis, badge pesanan, dan tombol refresh.



Gambar 7.3 Daftar Produk dari GET API

Daftar Produk menampilkan nama, kategori, SKU, harga, stok, status ketersediaan, pencarian, dan filter kategori.

← Detail Produk

Marimas

Tidak ada deskripsi.

Kategori	Minuman
SKU	MNM-MARIMAS
Harga	Rp3.000
Stok	65 pcs
Status tersedia	Tersedia
Dapat dipesan	Ya
Dibuat	11 Jul 2026, 21:28
Diperbarui	17 Jul 2026, 07:36

Gambar 7.4 Detail Produk

Detail Produk menampilkan kategori, SKU, harga, stok, status tersedia, status dapat dipesan, serta waktu pembuatan dan pembaruan.

←

Tambah Produk

Kategori

Makanan

Nama produk

Sotang Goreng

SKU

SKU akan dikirim dalam huruf kapital.

Deskripsi

enak dan lezat

Harga

Rp 10000

Stok

10

Satuan

pcs

Produk tersedia

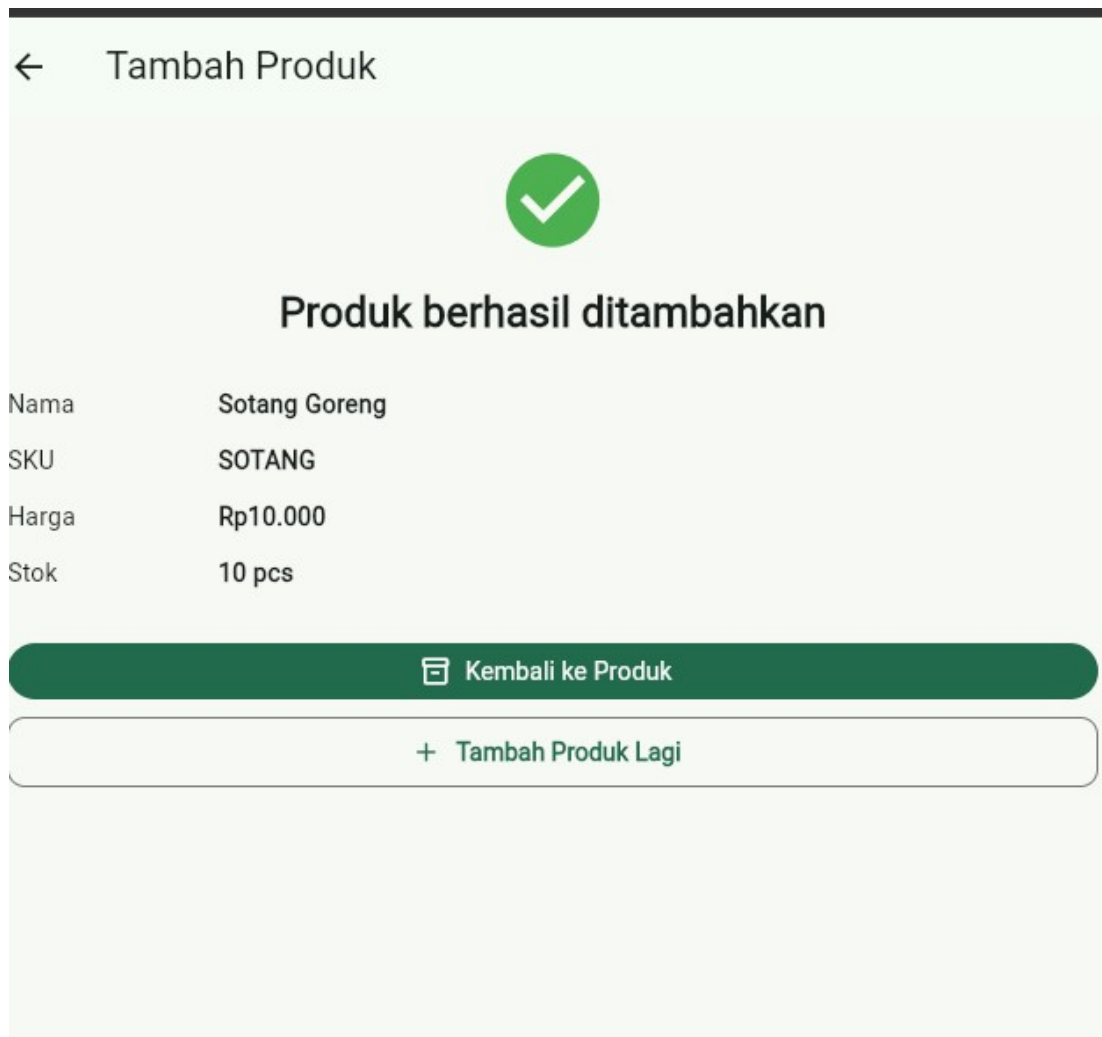
Produk dapat dipilih jika stok tersedia.

☒

Simpan Produk

Gambar 7.5 Form Tambah Produk

Form Tambah Produk menyediakan pilihan kategori, nama, SKU, deskripsi, harga, stok, satuan, dan status produk.



Gambar 7.6 Produk Berhasil Ditambahkan

Halaman sukses menampilkan nama, SKU, harga, dan stok produk yang telah disimpan melalui POST API.

Pesanan

KS

Cari pesanan

→

Status

Semua

Menunggu, dikonfirmasi, diproses, dan siap adalah status perlu tindakan.

7 pesanan

ORD-20260717-RJBJ9X

Meja 1

Nugraha Purnama

3 item

17 Jul 2026, 07:17

Menunggu

Rp10.000

ORD-20260716-R20YM1

Meja 1

Gusnik Bagus

1 item

16 Jul 2026, 21:25

Stok dikurangi

Selesai

Rp16.000

ORD-20260711-GWVQXW

Meja 1

gusnik

Selesai

Gambar 7.7 Daftar Pesanan dari GET API

Daftar Pesanan menampilkan nomor order, meja, pelanggan, jumlah item, waktu, status, total, pencarian, dan filter status.

28

←

Detail Pesanan

ORD-20260717-RJBJ9X

Menunggu

Meja

Meja 1

Pelanggan

Nugraha Purnama

Total

Rp10.000

Waktu

17 Jul 2026, 07:17

Status stok

Stok belum dikurangi

Catatan

jangan Pedas

Item Pesanan

Sempol

1 x Rp2.000

Rp2.000

Pop Ice

1 x Rp5.000

Rp5.000

Marimas

1 x Rp3.000

Rp3.000

Tindakan

✓ Konfirmasi Pesanan

✗ Batalkan Pesanan

Gambar 7.8 Detail Pesanan Menunggu

Detail pesanan pending menampilkan item, total, catatan, status stok, serta tombol Konfirmasi dan Batalkan.

←

Detail Pesanan

ORD-20260717-RJBJ9X

Dikonfirmasi

Meja

Meja 1

Pelanggan

Nugraha Purnama

Total

Rp10.000

Waktu

17 Jul 2026, 07:17

Status stok

Stok sudah dikurangi

Catatan

jangan Pedas

Item Pesanan

Sempol

1 x Rp2.000

Rp2.000

Pop Ice

1 x Rp5.000

Rp5.000

Marimas

1 x Rp3.000

Rp3.000

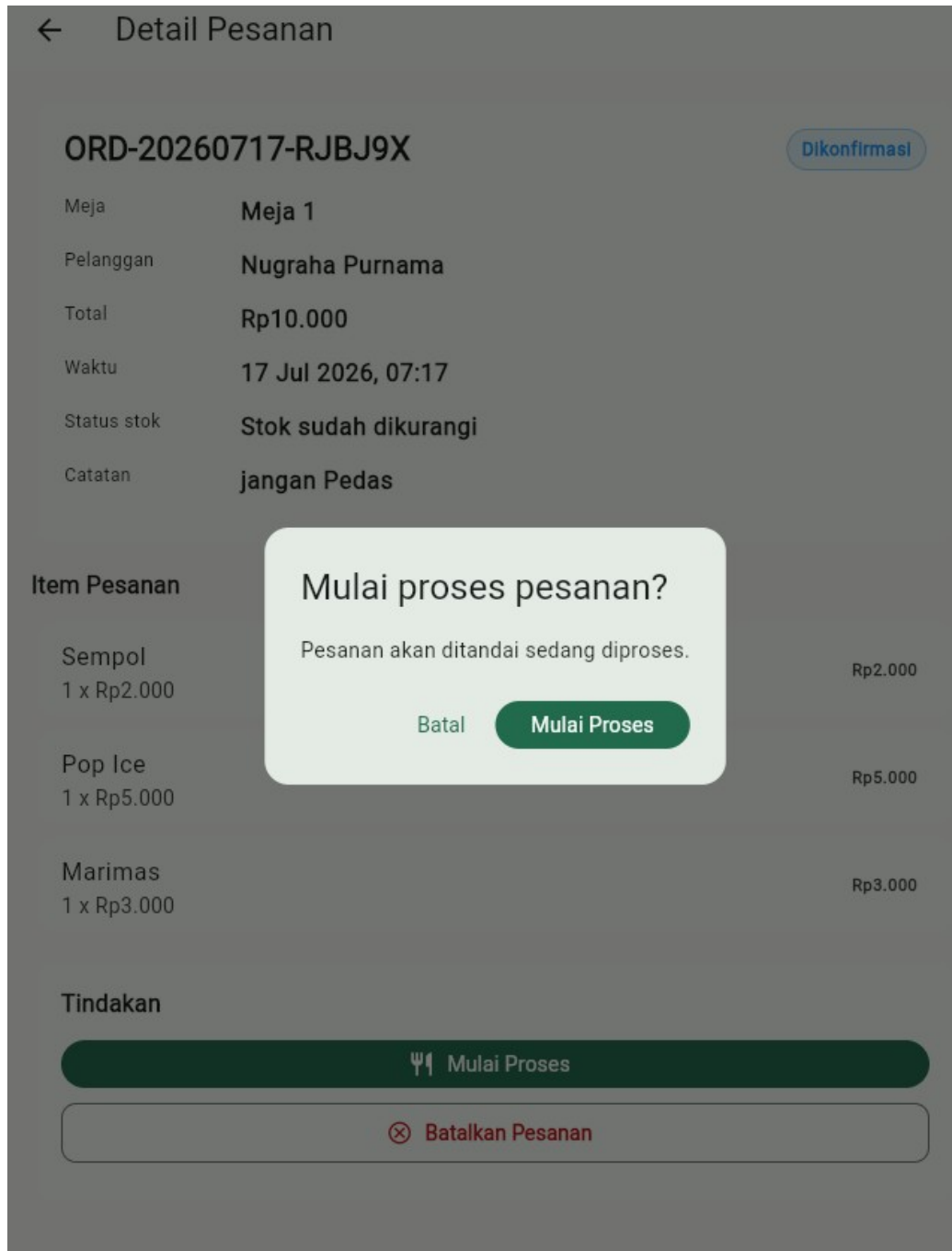
Tindakan

🍴 Mulai Proses

⊗ Batalkan Pesanan

Gambar 7.9 Pesanan Dikonfirmasi

Setelah konfirmasi, status berubah menjadi Dikonfirmasi dan informasi stok menunjukkan bahwa stok sudah dikurangi.



Gambar 7.10 Dialog Mulai Proses Pesanan

Dialog konfirmasi mencegah perubahan status secara tidak sengaja sebelum pesanan ditandai sedang diproses.

←

Detail Pesanan

ORD-20260717-RJBJ9X

Diproses

Meja

Meja 1

Pelanggan

Nugraha Purnama

Total

Rp10.000

Waktu

17 Jul 2026, 07:17

Status stok

Stok sudah dikurangi

Catatan

jangan Pedas

Item Pesanan

Sempol

1 x Rp2.000

Rp2.000

Pop Ice

1 x Rp5.000

Rp5.000

Marimas

1 x Rp3.000

Rp3.000

Tindakan

✓ Tandai Siap

⊗ Batalkan Pesanan

Gambar 7.11 Pesanan Sedang Diproses

Status Diproses menampilkan tombol Tandai Siap dan opsi pembatalan sesuai aturan transisi status.

←

Pembayaran Pesanan

ORD-20260717-RJBJ9X

Meja

Pelanggan

Total

Pembayaran

Meja 1

Nugraha Purnama

Rp10.000

Tunai

Item Pesanan

Sempol

1 x Rp2.000

Rp2.000

Pop Ice

1 x Rp5.000

Rp5.000

Marimas

1 x Rp3.000

Rp3.000

Uang dibayar

Rp 100000

Kembalian sementara

Rp90.000

Simpan Pembayaran

Gambar 7.12 Form Pembayaran Pesanan

Pembayaran Pesanan menampilkan item, total, input uang dibayar, kembalian sementara, dan metode pembayaran tunai.



Pembayaran berhasil

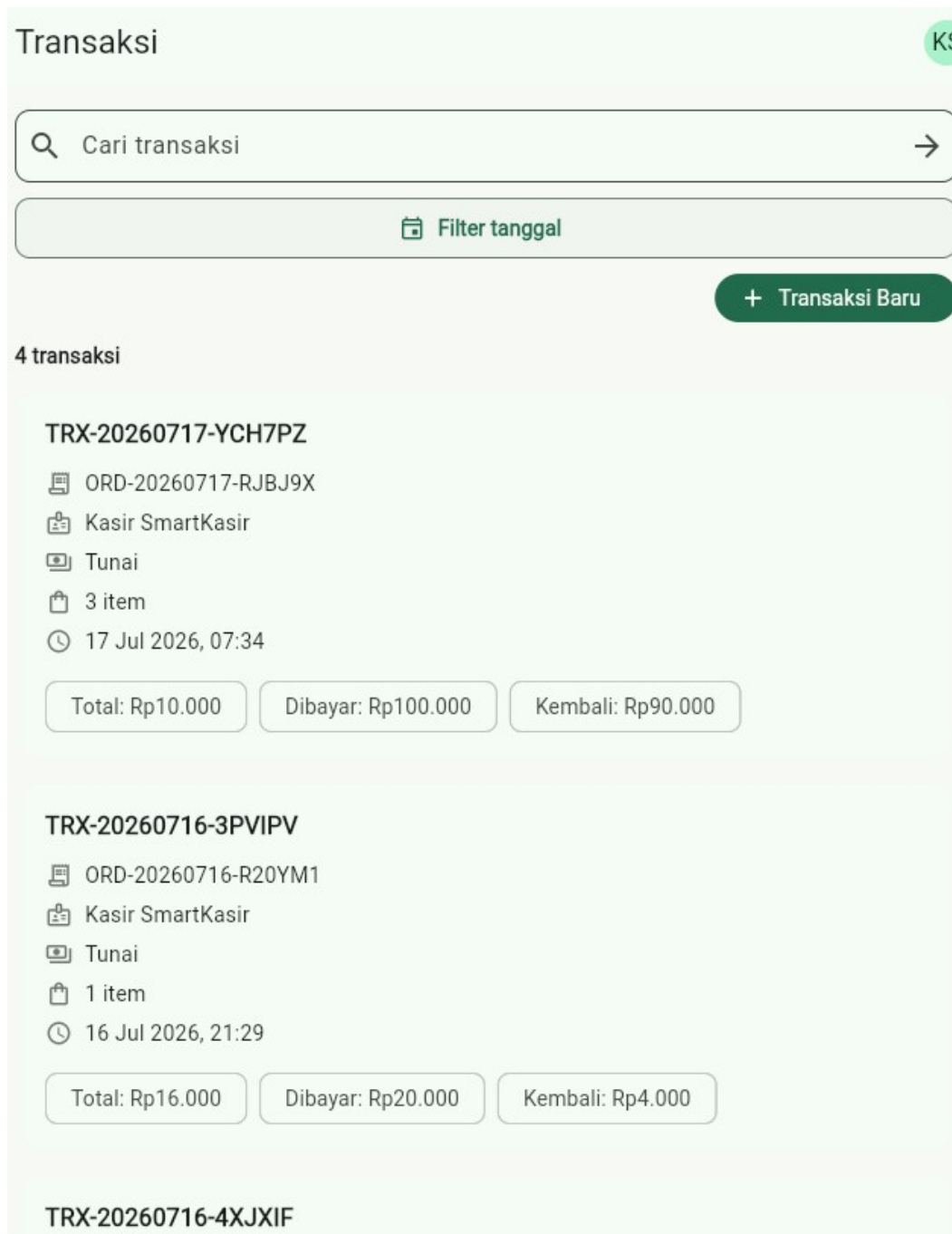
Nomor transaksi	TRX-20260717-YCH7PZ
Nomor pesanan	ORD-20260717-RJB9X
Total	Rp10.000
Dibayar	Rp100.000
Kembalian	Rp90.000
Kasir	Kasir SmartKasir
Pembayaran	Tunai

 [Kembali ke Pesanan](#)

 [Lihat Transaksi](#)

Gambar 7.13 Pembayaran Pesanan Berhasil

Halaman sukses menampilkan nomor transaksi, nomor pesanan, total, jumlah dibayar, kembalian, kasir, dan metode pembayaran.



Gambar 7.14 Daftar Transaksi dari GET API

Daftar Transaksi menampilkan transaksi langsung dan pembayaran order beserta total, pembayaran, kembalian, waktu, dan jumlah item.

← Detail Transaksi

TRX-20260717-5PUMC9

Pesanan	Transaksi Langsung
Kasir	Kasir SmartKasir
Total	Rp11.000
Dibayar	Rp20.000
Kembalian	Rp9.000
Pembayaran	Tunai
Waktu	17 Jul 2026, 07:36

Item Transaksi

Kentang Goreng 1 x Rp8.000	Rp8.000
Marimas 1 x Rp3.000	Rp3.000

Gambar 7.15 Detail Transaksi

Detail transaksi menampilkan nomor transaksi, jenis transaksi, kasir, total, pembayaran, kembalian, metode, waktu, dan item.

←

Transaksi Baru

Piscok

Rp3.000 | Stok 50

+ Tambah

Pop Ice

Rp5.000 | Stok 58

+ Tambah

Sempol

Rp2.000 | Stok 68

+ Tambah

Sotang Goreng

Rp10.000 | Stok 10

+ Tambah

Teh Sisri

Rp3.000 | Stok 70

+ Tambah

Keranjang

Kentang Goreng

Rp8.000 | Rp8.000

⊖

1

⊕

🗑

Piscok

Rp3.000 | Rp3.000

⊖

1

⊕

🗑

Jenis produk

2

Jumlah item

2

Total sementara

Rp11.000

Uang dibayar

Rp 20000

Kembalian sementara

Rp9.000

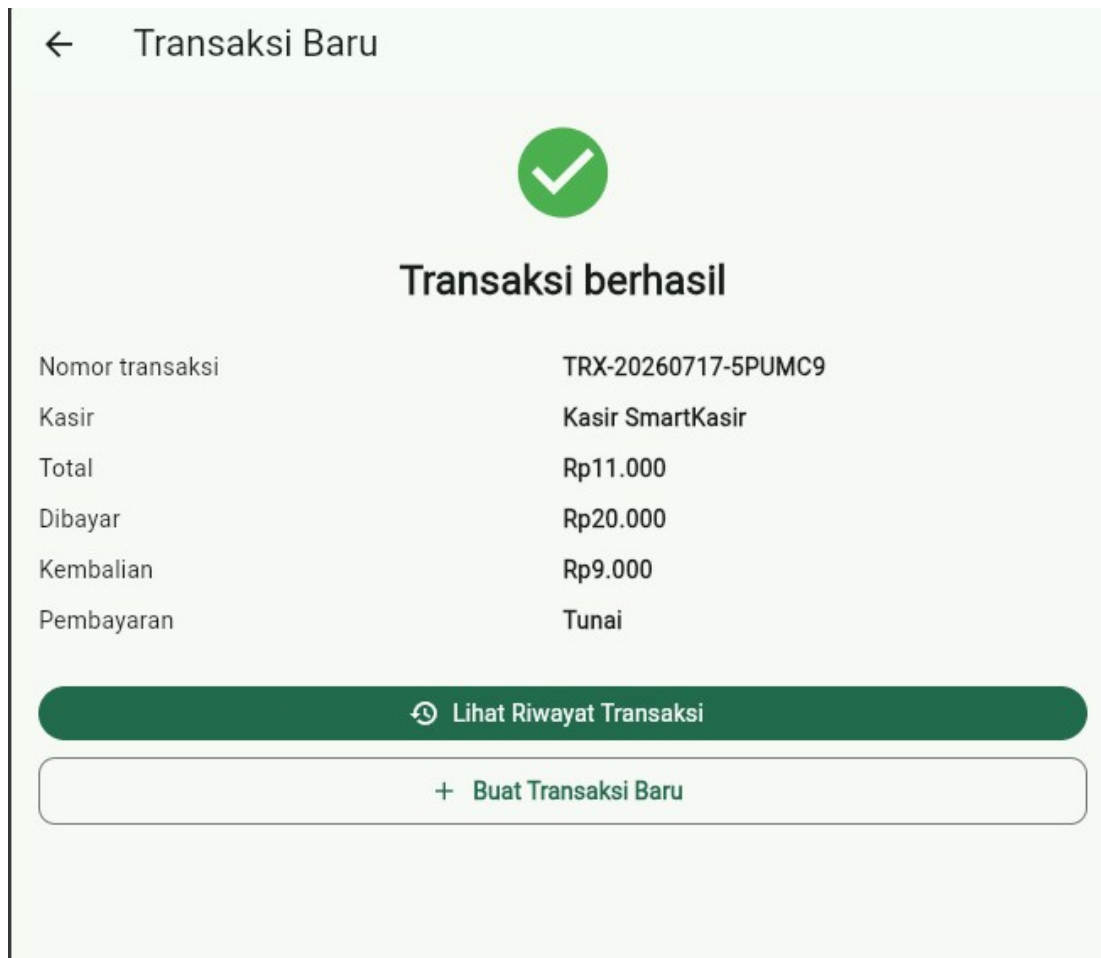
Metode pembayaran

Tunai

💰 Simpan Transaksi

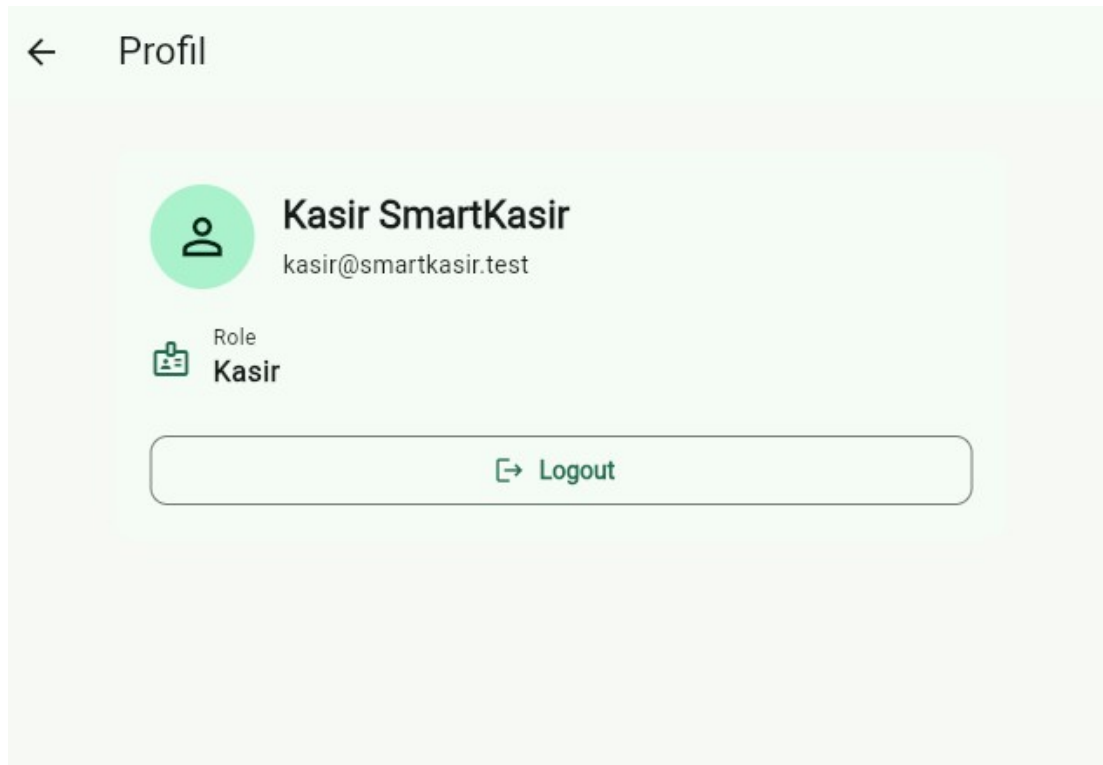
Gambar 7.16 Form Transaksi Kasir

Form transaksi menampilkan produk, keranjang, pengaturan jumlah, total sementara, uang dibayar, kembalian, dan tombol Simpan Transaksi.



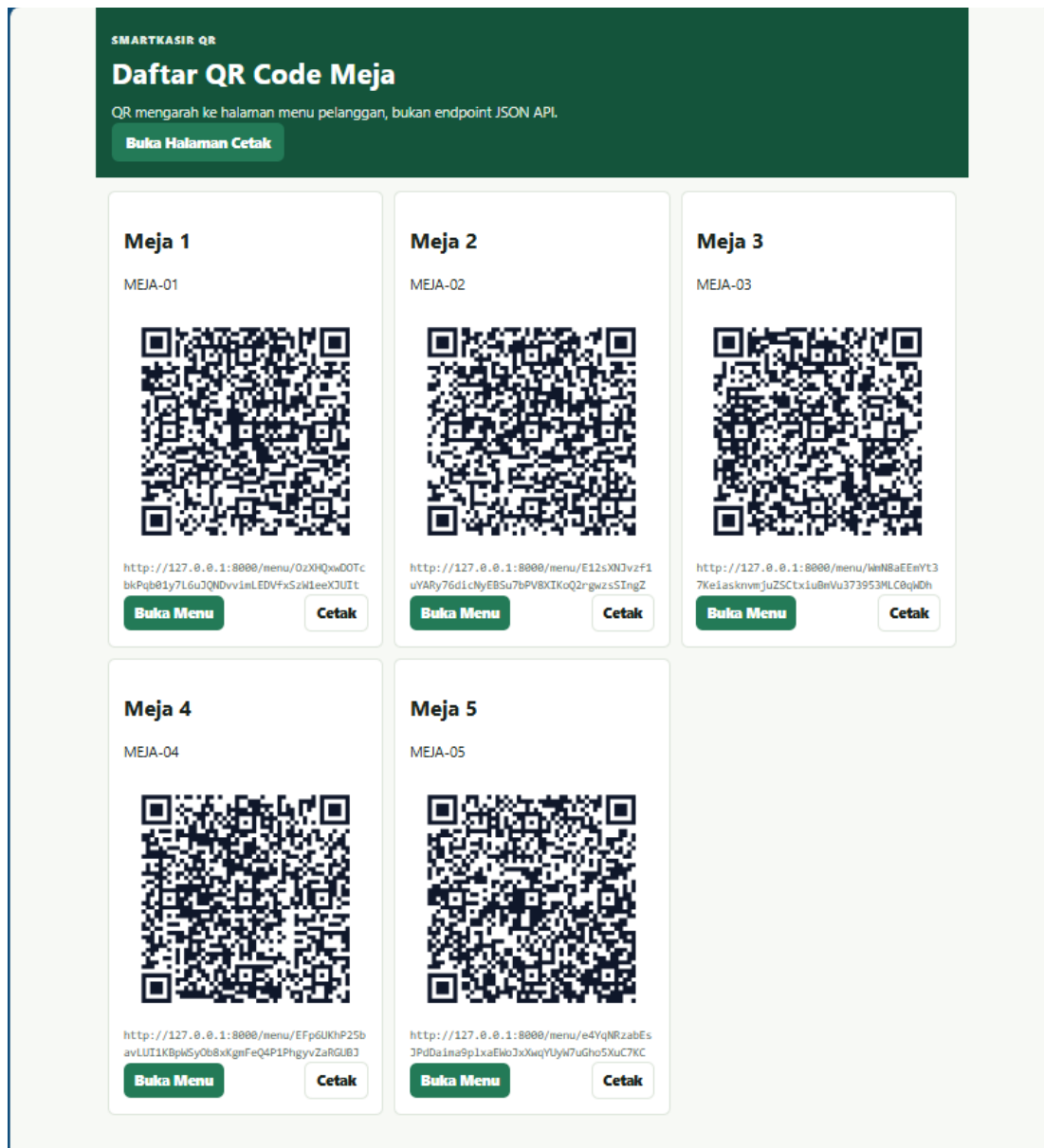
Gambar 7.17 Transaksi Kasir Berhasil

Halaman sukses menampilkan hasil final yang dihitung backend, termasuk nomor transaksi, total, pembayaran, dan kembalian.



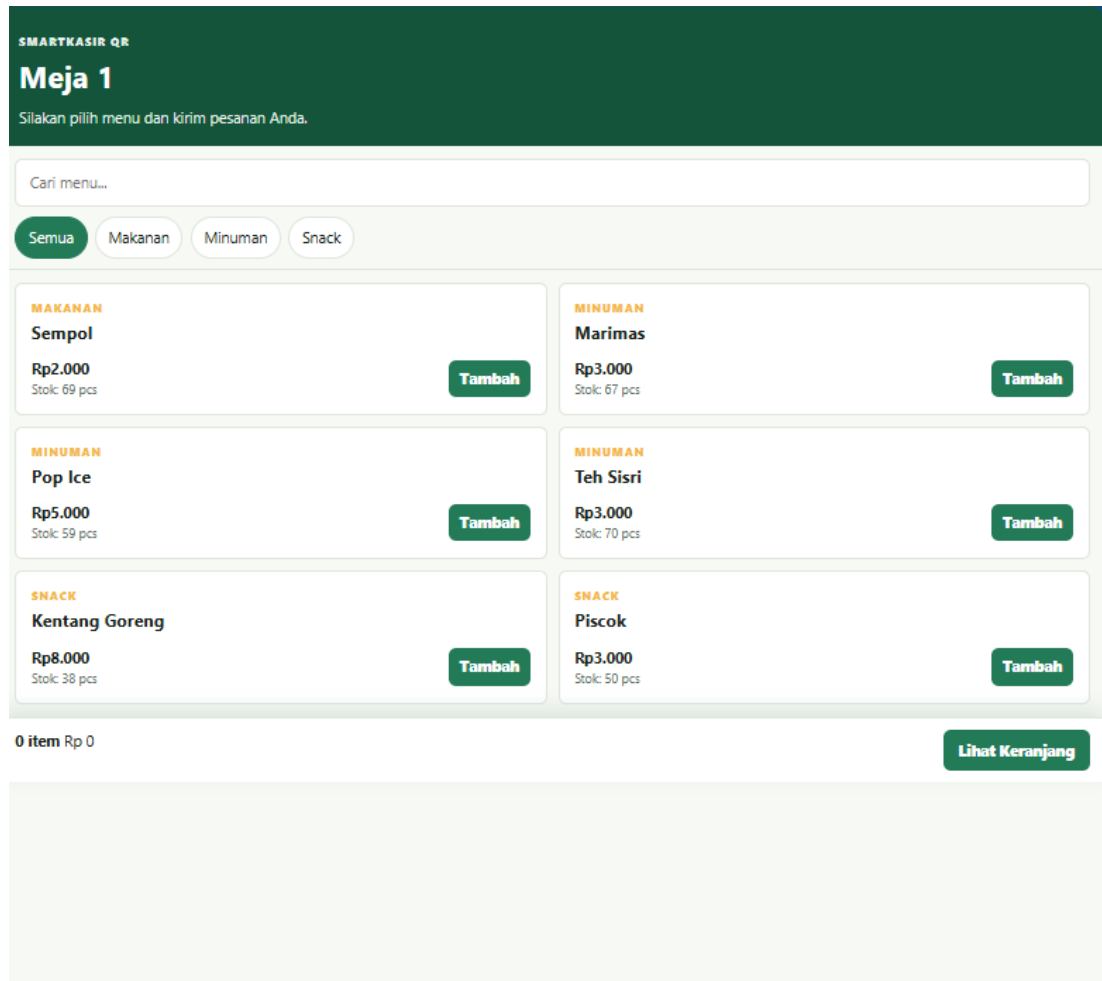
Gambar 7.18 Profil dan Logout

Halaman Profil menampilkan pengguna aktif dan tombol Logout untuk menghapus token serta membersihkan state aplikasi.



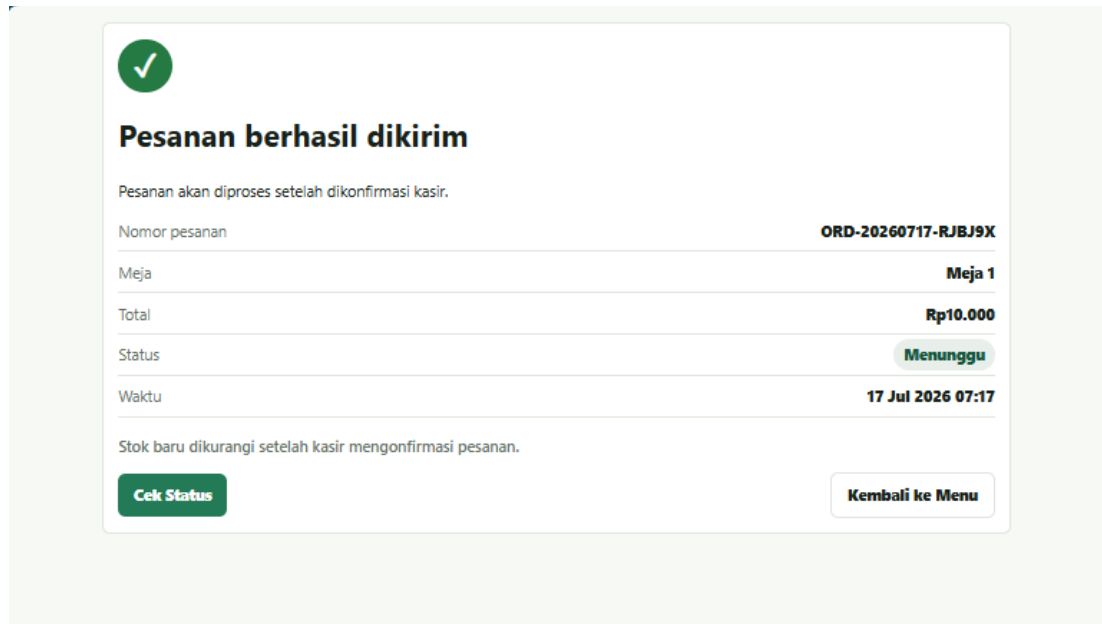
Gambar 7.19 Daftar QR Code Meja

Website administrator menampilkan QR Code dan link menu untuk setiap meja yang dapat dicetak atau dibuka.



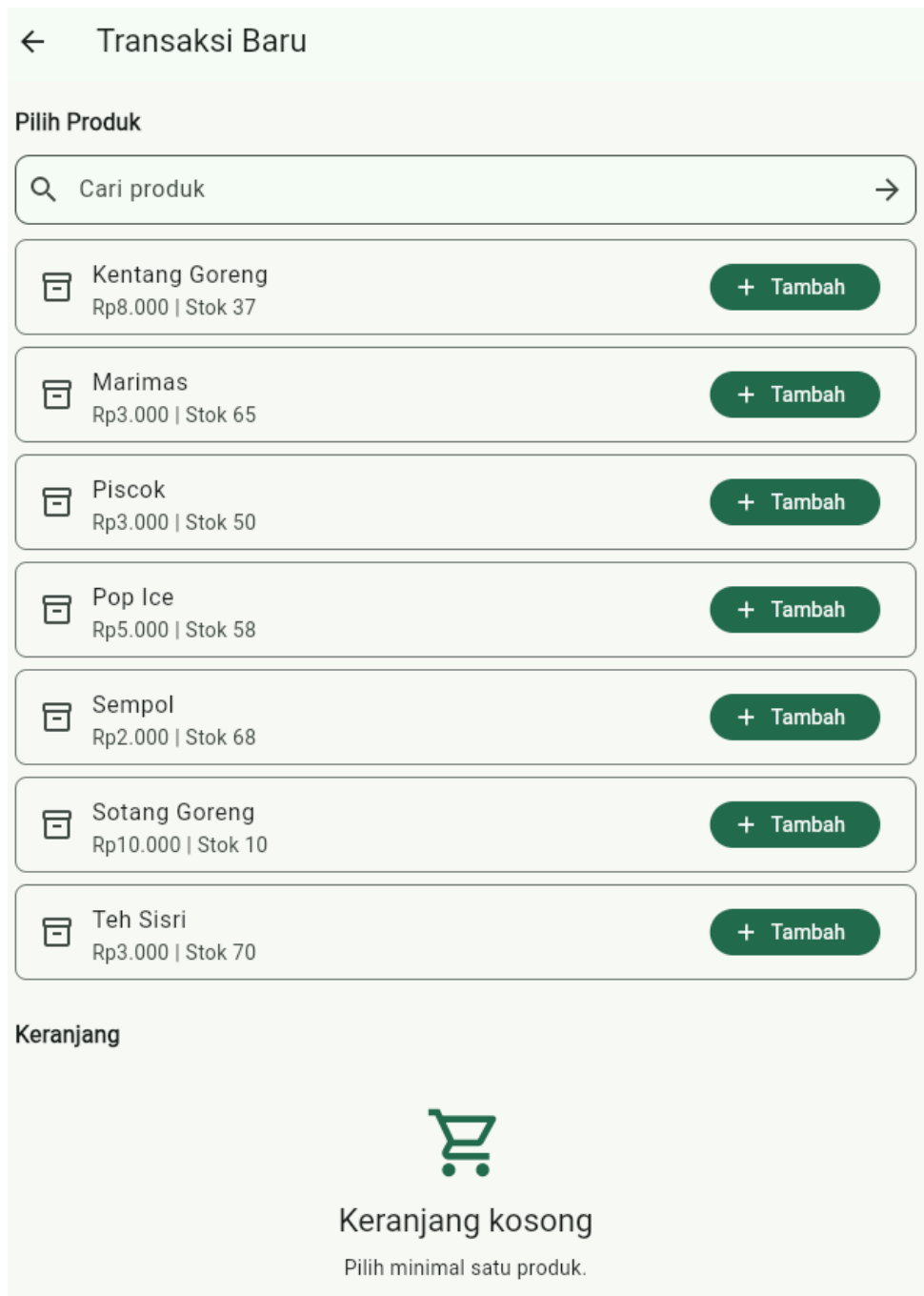
Gambar 7.20 Menu Pelanggan

Menu pelanggan menampilkan produk tersedia, kategori, pencarian, stok, tombol Tambah, dan akses ke keranjang.



Gambar 7.21 Pesanan Pelanggan Berhasil

Halaman sukses menampilkan nomor pesanan, meja, total, status Menunggu, waktu, dan informasi bahwa stok dikurangi setelah konfirmasi kasir.



Gambar 7.22 Empty State Keranjang Transaksi

Ketika belum ada produk dipilih, halaman transaksi menampilkan empty state yang memberi petunjuk kepada kasir untuk memilih minimal satu produk.

BAB VIII

CARA MENJALANKAN

8.1 Menjalankan Backend Laravel

Langkah berikut dijalankan dari folder backend. File .env dibuat dari .env.example, database SQLite disiapkan, migration dan seeder dijalankan, kemudian aset frontend dibangun.

Perintah Setup Laravel

Lokasi file: smartkasir-qr-api/

```
copy .env.example .env
composer install
php artisan key:generate
New-Item -ItemType File database\database.sqlite -Force
php artisan migrate --seed
npm install
npm run build
php artisan serve --host=127.0.0.1 --port=8000
```

8.2 Menjalankan Flutter Web

Perintah Flutter Web

Lokasi file: smartkasir-qr-mobile/

```
flutter pub get
flutter run -d edge --no-web-resources-cdn --dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

8.3 Build APK Android

Untuk Android Emulator, alamat 10.0.2.2 digunakan agar emulator dapat mengakses server Laravel yang berjalan pada komputer host. Untuk perangkat fisik, API_BASE_URL disesuaikan dengan IPv4 komputer atau menggunakan adb reverse.

Perintah Build APK Release

Lokasi file: smartkasir-qr-mobile/

```
flutter build apk --release --dart-define=API_BASE_URL=http://10.0.2.2:8000/api
```

8.4 Akun Demo

Akun Kasir Demo

Lokasi file: Login

```
Email: kasir@smartkasir.test
Password: password
```

8.5 Tautan Source Code

Tautan repository Flutter dan Laravel dicantumkan pada berkas pengumpulan atau disampaikan melalui tautan GitHub setelah proses push selesai.

BAB IX

PENUTUP

9.1 Kesimpulan

SmartKasir QR berhasil menghubungkan aplikasi Flutter dengan REST API Laravel. Sistem menerapkan login email dan password, autentikasi Bearer Token, penyimpanan token menggunakan flutter_secure_storage, halaman Home, Bottom Navigation empat menu, tiga halaman GET API, dan dua halaman POST API.

Website pelanggan melalui QR meja terintegrasi dengan aplikasi kasir. Alur pending, confirmed, processing, ready, completed, dan cancelled telah diterapkan. Sinkronisasi stok berjalan sesuai aturan: stok tidak berkurang saat pending, berkurang saat confirm, tidak berkurang kembali saat pembayaran, dan dikembalikan ketika pesanan dibatalkan setelah confirm.

Pengujian Flutter menghasilkan 89 test lulus tanpa masalah pada flutter analyze. Pengujian Laravel menghasilkan 95 test dan 267 assertions lulus, Laravel Pint lulus, serta build aset frontend berhasil. Hasil tersebut menunjukkan bahwa fungsi utama aplikasi telah berjalan sesuai kebutuhan tugas.

9.2 Saran

- Menggunakan HTTPS dan hosting backend agar sistem dapat diakses dengan aman dari luar jaringan lokal.
- Menambahkan pembayaran non-tunai dan integrasi payment gateway.
- Menambahkan printer struk dan laporan penjualan periodik.
- Menambahkan push notification atau pembaruan pesanan real-time.
- Melakukan build dan pengujian APK release pada emulator serta perangkat Android fisik.